Linear Methods for Classification

Flavien De Bock

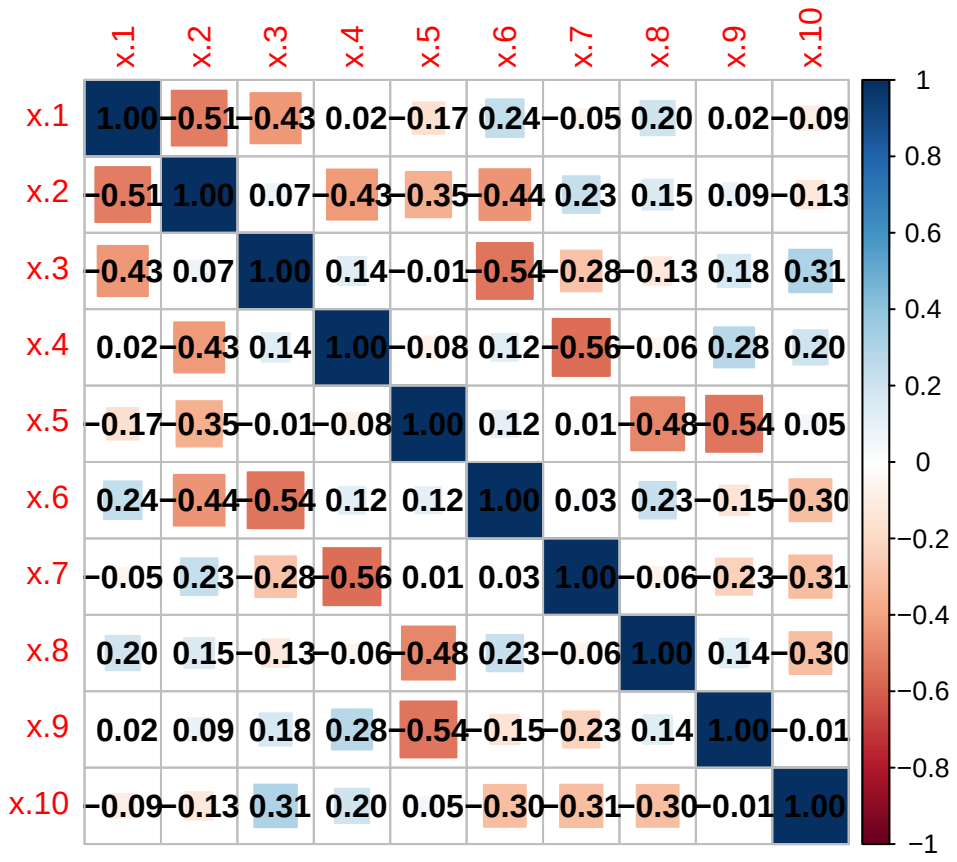
2026-08-26

```
rm(list = ls())
library(readr) ; library(tidyverse) ; library(corrplot) ; library(MASS)

train <- read.csv("vowel_train.csv", sep = ",")
train$x.10. <- gsub("\\\\", "", train$x.10.)
train$x.10. <- as.numeric(gsub("}", "", train$x.10.))
train$x.10 <- train$x.10. ; train$x.10. <- NULL

test <- read.csv("vowel_test.csv", sep = ",")
test$x.10. <- gsub("\\\\", "", test$x.10.)
test$x.10. <- as.numeric(gsub("}", "", test$x.10.))
test$x.10 <- test$x.10. ; test$x.10. <- NULL
head(train) ; summary(train)
```

```
##   y    x.1   x.2    x.3   x.4    x.5   x.6    x.7    x.8    x.9   x.10
## 1 1 -3.639 0.418 -0.670 1.779 -0.168 1.627 -0.388  0.529 -0.874 -0.814
## 2 2 -3.327 0.496 -0.694 1.365 -0.265 1.933 -0.363  0.510 -0.621 -0.488
## 3 3 -2.120 0.894 -1.576 0.147 -0.707 1.559 -0.579  0.676 -0.809 -0.049
## 4 4 -2.287 1.809 -1.498 1.012 -1.053 1.060 -0.567  0.235 -0.091 -0.795
## 5 5 -2.598 1.938 -0.846 1.062 -1.633 0.764  0.394 -0.150  0.277 -0.396
## 6 6 -2.852 1.914 -0.755 0.825 -1.588 0.855  0.217 -0.246  0.238 -0.365
```

```
##        y           x.1              x.2               x.3
##  Min.   : 1   Min.   :-5.211   Min.   :-1.2740   Min.   :-2.48700
##  1st Qu.: 3   1st Qu.:-3.923   1st Qu.: 0.9167   1st Qu.:-0.94550
##  Median : 6   Median :-3.097   Median : 1.7330   Median :-0.50250
##  Mean   : 6   Mean   :-3.167   Mean   : 1.7353   Mean   :-0.44800
##  3rd Qu.: 9   3rd Qu.:-2.512   3rd Qu.: 2.4038   3rd Qu.: 0.04925
##  Max.   :11   Max.   :-0.941   Max.   : 5.0740   Max.   : 1.41300
##       x.4               x.5               x.6               x.7
##  Min.   :-1.4090   Min.   :-2.1270   Min.   :-0.8360   Min.   :-1.53700
##  1st Qu.:-0.0835   1st Qu.:-0.9307   1st Qu.: 0.1085   1st Qu.:-0.29700
##  Median : 0.4565   Median :-0.4170   Median : 0.5275   Median : 0.04000
##  Mean   : 0.5250   Mean   :-0.3893   Mean   : 0.5850   Mean   : 0.01748
##  3rd Qu.: 1.1640   3rd Qu.: 0.1155   3rd Qu.: 1.0097   3rd Qu.: 0.34800
##  Max.   : 2.1910   Max.   : 1.8310   Max.   : 2.3270   Max.   : 1.40300
##       x.8                x.9               x.10
##  Min.   :-1.29300   Min.   :-1.6130   Min.   :-1.68000
##  1st Qu.:-0.01825   1st Qu.:-0.6737   1st Qu.:-0.50700
##  Median : 0.47700   Median :-0.2550   Median :-0.08250
##  Mean   : 0.41739   Mean   :-0.2681   Mean   :-0.08457
##  3rd Qu.: 0.86125   3rd Qu.: 0.1375   3rd Qu.: 0.30100
##  Max.   : 1.67300   Max.   : 1.3090   Max.   : 1.39600
```

```r
corr_matrix <- cor(train[,-1]) # correlation matrix
corrplot(corr_matrix, method = "square", addCoef.col = 'black')  # associated plot
```




```r
train[paste0("y_", 1:11)] <- lapply(1:11, function(k) { as.integer(train$y == k)})
test[paste0("y_", 1:11)] <- lapply(1:11, function(k) { as.integer(test$y == k)})

x_vars <- paste0("x.", 1:10) # string vector of the name of the features

# Regression Indicator Matrix ------

models <- vector("list", 11) # create an empty list to store the 11 regressions

for (k in 1:11) { # loop through the 11 groups
  y_var <- paste0("y_", k) # string vector of the name of group k

  formula_k <- as.formula(
    paste(y_var, "~", paste(x_vars, collapse = " + ")) # formula regress y_k on the features
  )

  models[[k]] <- lm(formula_k, data = train) # fit the reg
}

fitted_train <- as.data.frame(
  lapply(models, fitted) # retrieve the fitted values for each model
) # we store them in a data.frame
```

```r
names(fitted_train) <- paste0("y_", 1:11) # rename the columns for clarity

# the predicted outcome is the one with the highest fitted values
prediction_train <- max.col(fitted_train, ties.method = "first")

# Training error rate is ~0.48
train_error_rate <- mean(train$y != prediction_train) ; train_error_rate
```

```
## [1] 0.4772727
```

```r
# we obtain the fitted values of the model on the test data
fitted_test <- as.data.frame(
  lapply(models, function(model) predict(model, newdata = test))
)

names(fitted_test) <- paste0("y_", 1:11)

prediction_test <- max.col(fitted_test, ties.method = "first")

# the test error rate is ~0.67
test_error_rate <- mean(test$y != prediction_test) ; test_error_rate
```

```
## [1] 0.6666667
```

```r
# LDA ------

lda_model <- lda(y ~ x.1 + x.2 + x.3 + x.4 + x.5 + x.6 + x.7 + x.8 + x.9 + x.10,
                 data = train) ; lda_model
```

```
## Call:
## lda(y ~ x.1 + x.2 + x.3 + x.4 + x.5 + x.6 + x.7 + x.8 + x.9 +
##     x.10, data = train)
##
## Prior probabilities of groups:
##          1          2          3          4          5          6          7
## 0.09090909 0.09090909 0.09090909 0.09090909 0.09090909 0.09090909 0.09090909
##          8          9         10         11
## 0.09090909 0.09090909 0.09090909 0.09090909
##
## Group means:
##          x.1       x.2        x.3       x.4         x.5       x.6         x.7
## 1  -3.359563 0.0629375 -0.2940625 1.2033333  0.38747917 1.2218958  0.09637500
## 2  -2.708875 0.4906042 -0.5802292 0.8135000  0.20193750 1.0634792 -0.19091667
## 3  -2.440250 0.7748750 -0.7983958 0.8086667  0.04245833 0.5692500 -0.28006250
## 4  -2.226604 1.5258333 -0.8744375 0.4221458 -0.37131250 0.2483542 -0.01895833
## 5  -2.756312 2.2759583 -0.4657292 0.2253125 -1.03679167 0.3897917  0.23641667
## 6  -2.673542 1.7587708 -0.4745625 0.3505625 -0.66585417 0.4170000  0.16233333
## 7  -3.243729 2.4683542 -0.1050625 0.3964583 -0.98029167 0.1623125  0.01958333
## 8  -4.051333 3.2339792 -0.1739792 0.3965833 -1.04602083 0.1951875  0.08666667
## 9  -3.876896 2.3450208 -0.3668333 0.3170417 -0.39450000 0.8033750  0.02504167
## 10 -4.506146 2.6885625 -0.2849167 0.4695625 -0.03879167 0.6388750  0.13916667
## 11 -2.990396 1.4638750 -0.5098125 0.3716458 -0.38039583 0.7250417 -0.08339583
##           x.8         x.9        x.10
## 1  0.03710417 -0.62435417 -0.16162500
## 2  0.37381250 -0.51595833  0.08060417
```

```
## 3  0.20495833 -0.47827083  0.18187500
## 4  0.10714583 -0.32627083 -0.05375000
## 5  0.42462500 -0.20070833 -0.28070833
## 6  0.22925000 -0.20750000  0.05270833
## 7  0.76229167 -0.03027083 -0.12239583
## 8  0.82077083  0.10445833  0.02122917
## 9  0.73614583 -0.23183333 -0.14810417
## 10 0.38756250 -0.11102083 -0.27335417
## 11 0.50766667 -0.32750000 -0.22672917
##
## Coefficients of linear discriminants:
##               LD1        LD2        LD3         LD4         LD5        LD6
## x.1  -0.904263484 -1.0765784  0.2138645  0.45014309  0.01568403  1.3197512
## x.2   1.150256514 -0.3481586 -0.1219356  0.81974744 -0.07879069  1.2828586
## x.3   0.539113617  0.4775768  1.3152720 -0.18174531 -0.01428301  1.0421541
## x.4   0.024636593  0.6124198  0.3333835  0.89249728  1.19422174  0.2840921
## x.5  -0.007828209  1.6078817 -1.2374917  0.96925109  0.17984293  0.9791226
## x.6   0.708040285  1.4820258  0.8767861 -0.06187384 -0.84899844  1.9958303
## x.7   0.843500553  0.8675407  1.4607005  1.10382558  1.35657038  1.0703157
## x.8   1.305220749  0.9393954 -0.9967222 -1.05415855  0.85277476 -0.1212728
## x.9   0.965050787  0.5131795 -0.6927861  0.60778087 -0.04667900  0.5487891
## x.10  0.352677857  0.1517412 -0.7092612 -0.22606164  1.04293623  1.7009401
##             LD7         LD8         LD9        LD10
## x.1  -1.2563534  0.55370976 -0.21893931  0.01475366
## x.2  -0.8543263  0.85921988 -0.46773378 -0.20534101
## x.3  -1.3953387 -0.05664837 -0.58464728 -0.52218214
## x.4  -0.6079608  1.18671349 -0.26235352  0.22298959
## x.5  -1.5595892  0.12322124  0.38252584  0.05017199
## x.6  -0.2001485  0.54892997 -0.82835032  0.13158053
## x.7  -0.5029121  0.09927612  1.50030138  0.36516484
## x.8  -0.6617860  0.25729462  1.12742075  0.30135489
## x.9  -0.5244975 -1.05461865 -0.04187258  1.31351481
## x.10  0.5629667 -0.29898796  0.14972818 -0.23691329
##
## Proportion of trace:
##    LD1    LD2    LD3    LD4    LD5    LD6    LD7    LD8    LD9   LD10
## 0.5617 0.3518 0.0445 0.0191 0.0107 0.0083 0.0026 0.0011 0.0001 0.0001
```

```
lda_prediction_train <- predict(lda_model, newdata = train)
lda_prediction_test <- predict(lda_model, newdata = test)

# training error rate is ~ 0.32
lda_train_error_rate <- mean(train$y != lda_prediction_train$class) ; lda_train_error_rate
```

```
## [1] 0.3162879
```

```
# test error rate is ~ 0.56
lda_test_error_rate <- mean(test$y != lda_prediction_test$class) ; lda_test_error_rate
```

```
## [1] 0.5562771
```

```
# QDA ---------

qda_model <- qda(y ~ x.1 + x.2 + x.3 + x.4 + x.5 + x.6 + x.7 + x.8 + x.9 + x.10,
                 data = train) ; qda_model
```

```
## Call:
## qda(y ~ x.1 + x.2 + x.3 + x.4 + x.5 + x.6 + x.7 + x.8 + x.9 +
##     x.10, data = train)
##
## Prior probabilities of groups:
##          1          2          3          4          5          6          7
## 0.09090909 0.09090909 0.09090909 0.09090909 0.09090909 0.09090909 0.09090909
##          8          9         10         11
## 0.09090909 0.09090909 0.09090909 0.09090909
##
## Group means:
##          x.1       x.2        x.3       x.4         x.5       x.6         x.7
## 1  -3.359563 0.0629375 -0.2940625 1.2033333  0.38747917 1.2218958  0.09637500
## 2  -2.708875 0.4906042 -0.5802292 0.8135000  0.20193750 1.0634792 -0.19091667
## 3  -2.440250 0.7748750 -0.7983958 0.8086667  0.04245833 0.5692500 -0.28006250
## 4  -2.226604 1.5258333 -0.8744375 0.4221458 -0.37131250 0.2483542 -0.01895833
## 5  -2.756312 2.2759583 -0.4657292 0.2253125 -1.03679167 0.3897917  0.23641667
## 6  -2.673542 1.7587708 -0.4745625 0.3505625 -0.66585417 0.4170000  0.16233333
## 7  -3.243729 2.4683542 -0.1050625 0.3964583 -0.98029167 0.1623125  0.01958333
## 8  -4.051333 3.2339792 -0.1739792 0.3965833 -1.04602083 0.1951875  0.08666667
## 9  -3.876896 2.3450208 -0.3668333 0.3170417 -0.39450000 0.8033750  0.02504167
## 10 -4.506146 2.6885625 -0.2849167 0.4695625 -0.03879167 0.6388750  0.13916667
## 11 -2.990396 1.4638750 -0.5098125 0.3716458 -0.38039583 0.7250417 -0.08339583
##           x.8         x.9        x.10
## 1  0.03710417 -0.62435417 -0.16162500
## 2  0.37381250 -0.51595833  0.08060417
## 3  0.20495833 -0.47827083  0.18187500
## 4  0.10714583 -0.32627083 -0.05375000
## 5  0.42462500 -0.20070833 -0.28070833
## 6  0.22925000 -0.20750000  0.05270833
## 7  0.76229167 -0.03027083 -0.12239583
## 8  0.82077083  0.10445833  0.02122917
## 9  0.73614583 -0.23183333 -0.14810417
## 10 0.38756250 -0.11102083 -0.27335417
## 11 0.50766667 -0.32750000 -0.22672917
```

```r
qda_prediction_train <- predict(qda_model, newdata = train)
qda_prediction_test <- predict(qda_model, newdata = test)

# training error rate is ~ 0.01
qda_train_error_rate <- mean(train$y != qda_prediction_train$class) ; qda_train_error_rate
```

```
## [1] 0.01136364
```

```r
# test error rate is ~ 0.53
qda_test_error_rate <- mean(test$y != qda_prediction_test$class) ; qda_test_error_rate
```

```
## [1] 0.5281385
```